Word Vectors-II

(Skip-gram Model as Neural Network)

DR. JASMEET SINGH,
ASSISTANT PROFESSOR,
CSED, TIET

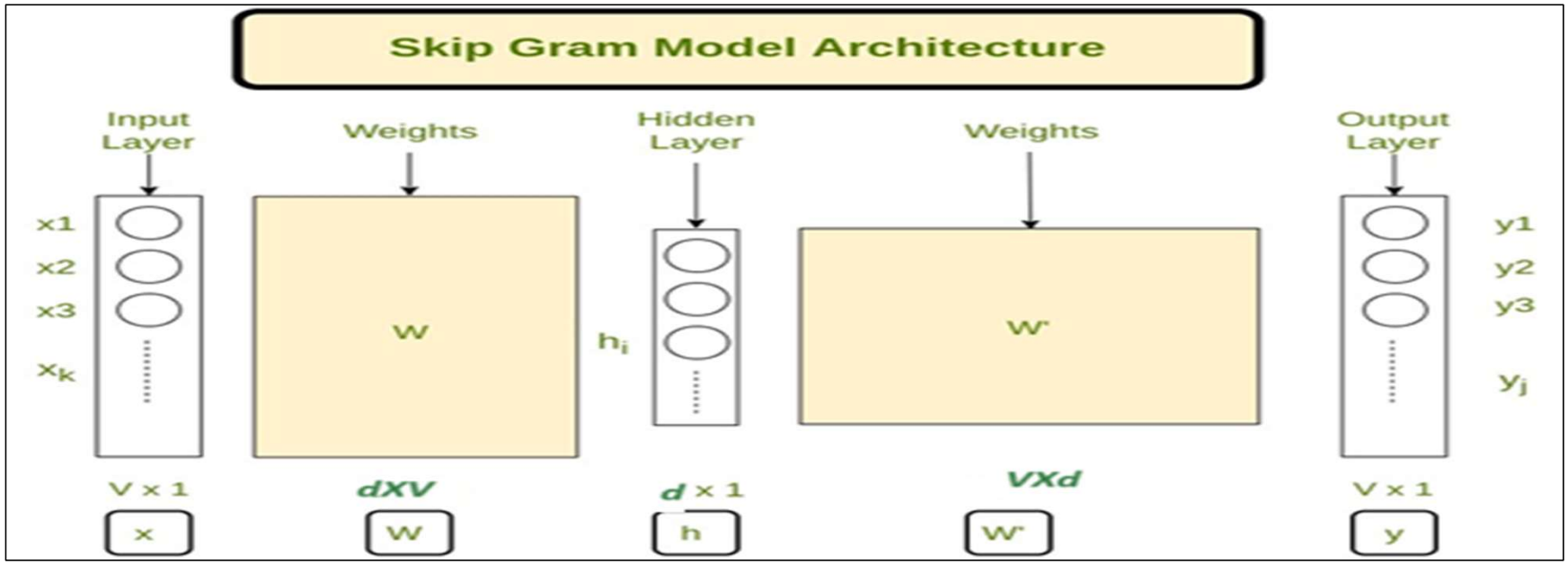
Skip-gram as Neural Network- Introduction

- Skip-gram model (SG) is computationally very intensive to train as we need to iterate over each text position i.e., t=1 to T (where T is generally very large for TBs/ PBs of data) and minimize the loss function for each contextual word given the target word at position t.

- In order to reduce the computational cost for training, the skip-gram word vector models can be modeled as a **two-layer feed-forward neural network**.

Skip-gram Neural Network Architecture

- Skip-gram model can be modeled using a shallow neural network with two layers. i.e. **a hidden layer and an output layer** (as shown below).



SG Neural Network – Training Data

- **Corpus:** The model is trained on a large text corpus, such as Wikipedia, news articles, or books.
- **Sliding Window**: A window of fixed size (e.g., 2, 5, or 10 words) is used to define context words around each target word.
- **Training Examples**: For each word in the corpus, the model generates training data in the form of (target word, context word) pairs. So, in one example target word is the input and context word is the output label.
- Both the target and the context word are encoded as one-hot vectors of vocabulary size (V). The target word is also called central word (usually denoted by c) and the contextual word is also called as outside/ true outside word (o).

SG Neural Network- Forward Propagation

Input Layer:

- The input is a one-hot encoded vector of size V (vocabulary size), representing the center word (c).
- For a word c (with vocabulary index, t), the one-hot vector is X where only the t^{th} element is 1, and the rest are 0s.

SG Neural Network- Forward Propagation (Contd..)

Hidden Layer:

- The hidden layer consists of d neurons, where d is the embedding size.
- The weight matrix between the input and hidden layer is W of size $d \times V$, where each column corresponds to the embedding of a word of dimensions d.
- There is no activation function or bias matrix used in the hidden layer. The output is simply the weighted sum of inputs.

$$i.e., Y = WX$$

- Since X is one-hot, this simply selects the t-th column of W, corresponding to the embedding central word (v_c).

$$Therefore, Y = W[:, t] = v_c \ (with\ shape\ dX1)$$

SG Neural Network- Forward Propagation (Contd..)

Output Layer:

- The output layer consist of V neurons and transforms the hidden layer output v_c into a probability distribution over the vocabulary.
- The weight matrix between the hidden and output layer is W' of size $V \times d$, where each row corresponds to the embedding of a word in the output space (u vectors of each word).
- We don't use any bias matrix in output layer, but use softmax as the activation function.

$$Hence, Z = W'Y = W'v_c \ (with\ shape\ of\ Z\ being\ VX1)$$

- The output logits (predicted values) are computed as the softmax function to Z to get the probabilities:

$$Predicted\ Values = \hat{Y}\ or\ S = softmax(Z)(with\ shape\ of\ S\ being\ VX1)$$

- For all $w \in \{1,\ldots,V\}$, this gives $s_w = softmax(z_w) = \frac{\exp(z_w)}{\sum_{w' \in V} \exp(z_{w'})} = \frac{\exp(u_w.v_c)}{\sum_{w' \in V} \exp(u_{w'}.v_c)} = P(w|o)$

SG Neural Network- Loss Function

- For skip-gram model, **multi-class cross-entropy** function is used because given a central/target word the model has to predict the contextual word from the entire vocabulary.
- The cross-entropy loss measures the distance between the predicted probability distribution $y\hat{}$ (or s) and the true distribution y.
- For one training example,

$$L = -Y_w \log(\hat{Y_w})$$

- where,
  - Y_w: True probability one-hot vector of word w (1 for outside/true outside/ contextual word , 0 otherwise).
  - $\hat{Y_w}$: Predicted probability

SG Neural Network- Loss Function (Contd..)

- Since Y_w=1 only for w=o (true outside word) , this simplifies to:

$$L = -\log\left(\hat{y_o}\right) = -\log(s_o) = -\log\left(P(o|c)\right)$$

$$where\ s_o = \hat{y_o} = P(o|c) = \frac{\exp(z_o)}{\sum_{w \in V} \exp(z_w)} = \frac{\exp(u_o . v_c)}{\sum_{w \in V} \exp(u_w . v_c)}$$

SG Neural Network- Back Propagation

- During the back propagation phase, gradient of loss function w.r.t trainable parameters i.e. W' $\left(\frac{\partial L}{\partial W'} \text{ or } dW'\right)$ and W $\left(\frac{\partial L}{\partial W} \text{ or } dW\right)$ will be computed.

$$\text{Now, } L = -\log\left(\hat{y_o}\right) = -\log(s_o) = -\log\left(P(o|c)\right)$$

$$where\ s_o = \hat{y_o} = P(o|c) = \frac{\exp(z_o)}{\sum_{w \in V} \exp(z_w)} = \frac{\exp(u_o . v_c)}{\sum_{w \in V} \exp(u_w . v_c)}$$

$\frac{\partial L}{\partial W'}$ *will be computed as* :

$$\frac{\partial L}{\partial W'} = \frac{\partial L}{\partial s_o} . \frac{\partial s_o}{\partial Z} . \frac{\partial Z}{\partial W'} = First\ Factor * Second\ Factor * Third\ Factor \quad \ldots\ldots\ldots (1)$$

SG Neural Network- Back Propagation (Contd....)

$$\boldsymbol{First\ Factor} = \frac{\partial L}{\partial s_o} = \frac{-\partial(\log(s_o))}{\partial s_o} = \frac{-1}{s_o}$$

In order to compute **second factor i.e., $\frac{\partial s_o}{\partial z_w}$,** we need to consider two cases as follows:

Case I: when w = o (true outside word)

$$\frac{\partial s_o}{\partial z_o} = \frac{\partial \frac{\exp(z_o)}{\sum_{w \in V} \exp(z_w)}}{\partial z_o} = \frac{\sum_{w \in V} \exp(z_w) \frac{\partial \exp(z_o)}{\partial z_o} - \exp(z_o) \frac{\partial \sum_{w \in V} \exp(z_w)}{\partial z_o}}{(\sum_{w \in V} \exp(z_w))^2}$$

$$= \frac{\sum_{w \in V} \exp(z_w) * \exp(z_o) - \mathrm{ex}\ (z_o) * \exp(z_o)}{(\sum_{w \in V} \exp(z_w))^2} = \frac{\exp(z_o) * [\sum_{w \in V} \exp(z_w) - \exp(z_o)]}{(\sum_{w \in V} \exp(z_w))^2}$$

$$= \frac{\exp(z_o)}{\sum_{w \in V} \exp(z_w)} * \left(\frac{\sum_{w \in V} \exp(z_w)}{\sum_{w \in V} \exp(z_w)} - \frac{\exp(z_o)}{\sum_{w \in V} \exp(z_w)} \right) = \boldsymbol{s_o * (1 - s_o)}$$

SG Neural Network- Back Propagation (Contd....)

Case II: when w ≠ o i.e., x (x is any other word other than true outside word)

$$\frac{\partial s_o}{\partial z_x} = \frac{\partial \frac{\exp(z_o)}{\sum_{w \in V} \exp(z_w)}}{\partial z_x} = \frac{\sum_{w \in V} \exp(z_w) \frac{\partial \exp(z_o)}{\partial z_x} - \exp(z_o) \frac{\partial \sum_{w \in V} \exp(z_w)}{\partial z_x}}{(\sum_{w \in V} \exp(z_w))^2}$$

$$= \frac{\sum_{w \in V} \exp(z_w) * 0 - \exp(z_o) * \exp(z_x)}{(\sum_{w \in V} \exp(z_w))^2} = -\frac{\exp(z_o)}{\sum_{w \in V} \exp(z_w)} * \frac{\exp(z_x)}{\sum_{w \in V} \exp(z_w)} = -\boldsymbol{s_o} * \boldsymbol{s_x}$$

$$\boldsymbol{Thus}, \frac{\partial s_o}{\partial z_w} = \begin{cases} -\boldsymbol{s_o} * (\boldsymbol{s_o} - \boldsymbol{1}) \; \boldsymbol{if} \; \boldsymbol{w = o} \\ -\boldsymbol{s_o} * \boldsymbol{s_x} \; \; \boldsymbol{if} \; \boldsymbol{w \neq o} \end{cases}$$

Both, these cases can be combined as $\frac{\partial s_o}{\partial Z} = -\boldsymbol{s_o} * (\boldsymbol{S} - \boldsymbol{Y_w})$

where, y_w is true label and is equal to 1 for w=o and 0 otherwise.

SG Neural Network- Back Propagation (Contd....)

$$\boldsymbol{Third\ Factor} = \frac{\partial z_w}{\partial W'} = Y = v_c$$

Substituting First, Second, and Third Factor in equation (1)

$$\frac{\partial L}{\partial W'} = \frac{\partial L}{\partial s_o} . \frac{\partial s_o}{\partial z_w} . \frac{\partial z_w}{\partial W'} = First\ Factor * Second\ Factor * Third\ Factor$$

$$= -\frac{1}{s_o} * -\boldsymbol{s_o} * (\boldsymbol{S} - \boldsymbol{Y_w})\boldsymbol{v_C^T}$$

$$\boldsymbol{Thus,} \frac{\boldsymbol{\partial L}}{\boldsymbol{\partial W'}} = (\boldsymbol{S} - \boldsymbol{Y_w})\boldsymbol{v_C^T} = \begin{cases} (\boldsymbol{P(o|c)} - \boldsymbol{1}).\boldsymbol{v_C^T}\ \boldsymbol{if\ w = o} \\ \boldsymbol{P(x|c)}.\boldsymbol{v_C^T}\ \boldsymbol{if\ w = x\ (\neq o)} \end{cases}$$

SG Neural Network- Back Propagation (Contd....)

$Now, \frac{\partial L}{\partial W}$ *will be computed as*:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial s_o} \cdot \frac{\partial s_o}{\partial Z} \cdot \frac{\partial Z}{\partial Y} \cdot \frac{\partial Y}{\partial W}$$

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Z} \cdot \frac{\partial Z}{\partial Y} \cdot \frac{\partial Y}{\partial W} = Term1 \,.Term2.Term\ 3 \ldots\ldots\ldots (2)$$

$Term1 = \frac{\partial L}{\partial Z} = S - \boldsymbol{Y_w}$(shape is VX1) (computed in previous step)

$$Term2 = \frac{\partial Z}{\partial Y} = \frac{\partial W'Y}{\partial Y} = W'\ (shape\ is\ V \times d)$$

$$Term3 = \frac{\partial Y}{\partial W} = \frac{\partial WX}{\partial W} = X$$

SG Neural Network- Back Propagation (Contd....)

Substitute, Term1, Term2, Term3 in Equation 2:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Z}.\frac{\partial Z}{\partial Y}.\frac{\partial Y}{\partial W} = Term1\ .Term2.Term\ 3$$

$$\frac{\partial L}{\partial W} = W'^{T}.(S - Y).X^{T}$$

(Because gradient of W w.r.t L i.e., dW should be of shape dXV)

$$\frac{\partial L}{\partial W} = [u_1 \quad u_2 \quad u_3 \dots\dots\dots\dots\dots u_v].\left(\begin{bmatrix} P(w_1|c) \\ P(w_2|c) \\ P(w_1|c) \\ \vdots \\ \vdots \\ P(o|c) \\ \vdots \\ P(w_v|c) \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}\right).X^{T}$$

SG Neural Network- Back Propagation (Contd....)

$$\frac{\partial L}{\partial W} = [u_1\ P(w_1|c) + u_2\ P(w_2|c) + \ldots.\ u_o P(o|c) + \cdots u_v\ P(w_v|c) - u_o]\ .\ X^T$$

$$\frac{\partial L}{\partial W} = (\sum_{w \in V} P(w|c) u_w - u_o).\ X^T$$

Since X is only one-hot vector

$$\frac{\partial L}{\partial W} = (\sum_{w \in V} P(w|c) u_w - u_o).\ [0 \quad 0 \quad 0 \ldots 1 \ldots \ldots 0]$$

So only one column corresponding to central word will be updated.

$$\boldsymbol{\frac{\partial L}{\partial W_{[:,c]}} = \frac{\partial L}{\partial v_c} = (\sum_{w \in V} P(w|c) u_w - u_o)}$$